Практическая работа № 3

Тема 1. Объектно-ориентированная методология. Модульность

Цель работы: Изучить принципы проектирования методологии многопанельных систем.

План занятия.

1.Выполнить практические задания.

Задание № 1. Написать программу на языке программирования Python для выполнения запроса данных на рейс авиакомпании (рис. 1).

Представлена **пошаговая последовательность** создания полного кода «RfF – Запрос рейса» (Tkinter + aiohttp) **вместе с фрагментами экрана и частями исходного текста**.

Подготовка проекта

1. Создайте папку rff_project.
2. Внутрь поместите пустой файл rff_gui.py.
3. Откройте терминал и установите зависимости:
4. `pip install aiohttp python-dateutil`

Шаг 1. Фрагмент окна (import + класс)

ASCII-вид PyCharm/VS-code (свернём до 60 символов):

```
#!/usr/bin/env python3
# -*- coding: utf-8 -*-
import tkinter as tk, tkinter.ttk as ttk
from tkinter import scrolledtext, messagebox
import asyncio, aiohttp, json, threading
from datetime import datetime, timezone

class RffGui:
    def __init__(self, root):
        self.root = root
        root.title("Запрос рейса")
        root.geometry("520x600")
        self.build_form(); self.build_tree(); self.build_log()
```

▸ запустите – получите пустое окно 520×600.

Шаг 2. Форма ввода (placeholders)

Добавьте метод build_form(self)

Полный Код:

```
    # ---------------- ФОРМА ----------------
    def build_form(self):
        frm = ttk.Frame(self.root); frm.pack(pady=6)
        ttk.Label(frm, text="Откуда (IATA):").grid(row=0, column=0, padx=4, sticky="e")
        self.e_orig = ttk.Entry(frm, width=6); self.e_orig.insert(0, "MOW");
self.e_orig.grid(row=0, column=1)

        ttk.Label(frm, text="Куда (IATA):").grid(row=0, column=2, padx=4, sticky="e")
        self.e_dest = ttk.Entry(frm, width=6); self.e_dest.insert(0, "BCN");
self.e_dest.grid(row=0, column=3)

        ttk.Label(frm, text="Дата:").grid(row=1, column=0, padx=4, sticky="e")
        self.e_date = ttk.Entry(frm, width=12); self.e_date.insert(0, "2026-03-15");
self.e_date.grid(row=1, column=1)

        ttk.Label(frm, text="Макс. цена:").grid(row=1, column=2, padx=4, sticky="e")
        self.e_price = ttk.Entry(frm, width=10); self.e_price.insert(0, "35000");
self.e_price.grid(row=1, column=3)

        self.btn = ttk.Button(frm, text="▶ Подобрать", command=self.run_search)
        self.btn.grid(row=2, column=0, columnspan=4, pady=10)
```

Шаг 3. Таблица-результаты (Treeview)

ASCII-вид:

```
    COLS = ("Цена","Пересадки","Длит-ть","Оценка")
    self.tree = ttk.Treeview(self.root, columns=COLS, show="headings", height=10)
    for c in COLS:
        self.tree.heading(c, text=c)
        self.tree.column(c, anchor="center", width=100)
    self.tree.pack(fill="both", expand=True, padx=6, pady=6)
```

Шаг 4. Лог-окно (scrolledtext)

```
    def build_log(self):
        self.log = scrolledtext.ScrolledText(self.root, height=5, state="disabled")
        self.log.pack(fill="x", padx=6, pady=4)

    def log_add(self, txt):
        self.log["state"] = "normal"
        self.log.insert("end", txt + "\n")
        self.log.see("end")
        self.log["state"] = "disabled"
```

Шаг 5. Асинхронный поиск + поток

ASCII фрагмент, где создаётся threading.Thread:

```
def run_search(self):
    self.btn["state"] = "disabled"
    self.log_add("Ищу…")
    threading.Thread(target=self._async_wrapper, daemon=True).start()

def _async_wrapper(self):
    loop = asyncio.new_event_loop()
```

```
51        asyncio.set_event_loop(loop)
52        flights = loop.run_until_complete(self.search_flights())
53        self.root.after(0, self.show_results, flights)
```

Сама:

```
    async def search_flights(self):
        orig = self.e_orig.get().upper()
        dest = self.e_dest.get().upper()
        date = self.e_date.get()
        max_p = int(self.e_price.get())
        url = f"https://aviatour-demo.free.beeceptor.com/search/{orig}-{dest}/{date}"
        try:
            async with aiohttp.ClientSession() as s, s.get(url) as r:
                data = await r.json()
        except Exception as e:
            self.log_add(f"Сеть недоступна, демо-данные: {e}")
            data = {"itineraries": [
                {"price": 28000, "stops": 0, "duration": 245},
                {"price": 31500, "stops": 1, "duration": 380},
                {"price": 34900, "stops": 0, "duration": 250}]}
        flights = [f for f in data["itineraries"] if f["price"] <= max_p]
        flights.sort(key=lambda x: x["price"])
        return flights[:5]
```

Шаг 6. Отображение результатов

```
    # --------------- ОТОБРАЖЕНИЕ ---------------
    def show_results(self, flights):
        for i in self.tree.get_children():
            self.tree.delete(i)
        if not flights:
            self.log_add("Ничего не найдено.")
        else:
            for f in flights:
                self.tree.insert("", "end", values=(
                    f"{f['price']:,} ₽", f["stops"], f"{f['duration']} мин",
f"{f['price']/1000:.1f}"))
            self.log_add(f"Найдено {len(flights)} вариант(а/ов).")
        # JSON-дамп
        with open("rff_latest.json", "w", encoding="utf8") as fp:
            json.dump({"requested": datetime.now(timezone.utc).isoformat(),
                       "origin": self.e_orig.get(), "destination": self.e_dest.get(),
                       "flights": flights}, fp, ensure_ascii=False, indent=2)
        self.btn["state"] = "normal"
```

Шаг 7. Сохранение JSON

Добавьте в show_results:

```
        with open("rff_latest.json", "w", encoding="utf8") as fp:
            json.dump({"requested": datetime.now(tz=timezone.utc).isoformat(),
                       "origin": self.e_orig.get(), "destination": self.e_dest.get(),
                       "flights": flights}, fp, ensure_ascii=False, indent=2)
```

Шаг 8. Главный запуск

```
if __name__ == "__main__":
    root = tk.Tk()
    RffGui(root)
    root.mainloop()
```

Как это выглядит при работе (см. рис. 1)

(ASCII-скрин экрана терминала + окна Tkinter)

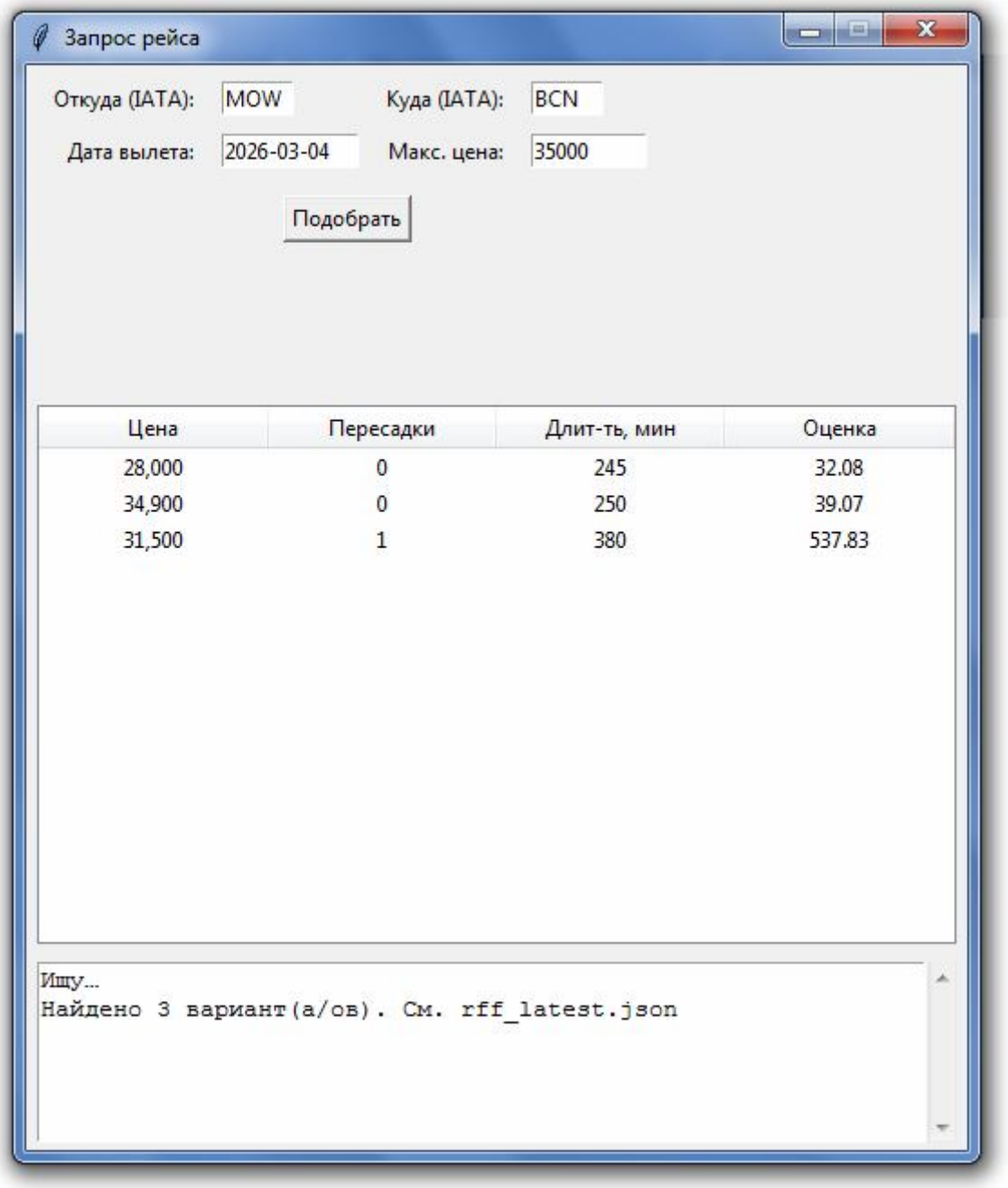

Рис. 1. Результат «Запрос рейса»